

NMAM INSTITUTE OF TECHNOLOGY, NITTE

Department of Information Science and Engineering

OPERATING SYSTEMS PROGRAMMING ASSIGNMENT REPORT

AI-Assisted Synchronization Algorithms with Program-Generated HTML Simulation

Readers-Writers Problem and Dining Philosophers Problem
(Semaphore-based and Monitor-based implementations in Python)

SUBMITTED BY

Name	Disha Kumari
Registration / USN	NNM24IS074
Course	Operating Systems
GitHub Repository	https://github.com/Disha-kumari/OS-Synchronization-Assignment

SUBMITTED TO

Dr Jason Elroy Martis
Associate Professor
Dept Information Science and Engineering, Nitte

1. Problem Statement and Objective

When several threads share data or resources, uncontrolled access can cause race conditions and incorrect results. This assignment implements two classic synchronization problems using two different synchronization approaches: semaphores and monitors.

The problems implemented are:

1. Readers-Writers using semaphores
2. Readers-Writers using a monitor
3. Dining Philosophers with four philosophers using semaphores
4. Dining Philosophers with four philosophers using a monitor

The objective is to demonstrate mutual exclusion, synchronization, deadlock prevention/avoidance, and starvation considerations. Each program also records execution events and generates an HTML simulation based on the actual execution trace.

Concurrent programs are hard to reason about because the scheduler, not the programmer, decides the order in which threads run, so the same program can behave differently from one run to the next. Semaphores, introduced by Dijkstra [6], and monitors, proposed by Hoare [8], are two classical tools for controlling this. The Readers-Writers problem [7] and the Dining Philosophers problem, also posed by Dijkstra, are the standard problems used to study them. All four programs in this project are written in Python using the standard threading module [1], and each one also records its own execution so that it can be replayed in the browser.

The assignment also focuses on understanding how synchronization mechanisms control access when several threads execute concurrently. Instead of only checking the final output, the programs record important execution events so that the behaviour of the threads can be observed. This makes it easier to understand when a thread is waiting, when it enters a critical section, and when it releases a shared resource.

The HTML simulation provides a visual representation of these events. This helps connect the theoretical concepts of operating-system synchronization with the actual execution of the Python programs. The same approach is applied to both synchronization problems so that the semaphore-based and monitor-based solutions can be compared.

2. Readers-Writers - Semaphore Solution

Shared resource: An integer `shared["value"]`. Readers read this value, while writers increment it.

Semaphores: `mutex = 1` protects the reader count and `rw_lock = 1` provides exclusive access to the shared resource[1].

The first reader acquires `rw_lock`, preventing writers from entering. The last reader releases `rw_lock`.

A writer acquires `rw_lock` exclusively before modifying the shared value.

Critical section: Access to the shared value. Multiple readers can access it together, but a writer must access it alone.

This implementation follows a readers-preference approach, so writers can theoretically wait for a long time if readers continue to arrive[2].

This is the classical first Readers-Writers solution described by Courtois et al. [7], built from the semaphore primitive introduced by Dijkstra [6]. The shared integer is deliberately updated in two steps: the writer reads the value, waits briefly, and then writes `value + 1`. If two writers were ever inside together, this short delay would make a lost update very likely, so the final check at the end of the program (`value` equals number of writers multiplied by rounds) works as a simple test of mutual exclusion. Reader and writer events are logged inside the critical section, so the recorded order matches the real order in which the threads entered it.

The main purpose of this implementation is to allow multiple readers to access the shared resource at the same time while ensuring that a writer receives exclusive access. The `mutex` semaphore protects the reader count so that two readers cannot incorrectly modify the count simultaneously. The `rw_lock` controls access to the actual shared resource.

When the first reader enters, it locks the resource for readers as a group. Other readers can then continue reading without requiring exclusive access to the shared value. When the last reader leaves, the resource is released so that a waiting writer can enter. This demonstrates how synchronization can allow concurrency where it is safe while still protecting the critical section.

The execution events recorded by the program also help verify this behaviour. The generated simulation can show readers entering together and writers waiting until the readers have finished. This provides a direct way to observe the synchronization rules during execution.

3. Readers-Writers - Monitor Solution

A class RWMonitor acts as the monitor using a lock and a condition variable.

The monitor maintains the number of active readers, whether a writer is active, and the number of waiting writers.

A reader waits if a writer is active or if writers are already waiting.

A writer waits while readers or another writer are active.

`notify_all()` is used to wake waiting threads when the shared resource becomes available.

The writer-preference design gives waiting writers priority over new readers and reduces the possibility of writer starvation.

In a monitor, every method runs while holding one lock, so only one thread can change the shared counters at a time [8]. A thread that cannot continue calls `wait()`, which releases the lock and puts the thread to sleep. When it wakes up, it checks its condition again inside a while loop, because another thread may have changed the state in the meantime. Using a while loop instead of an if statement is a standard rule for monitor code.

The monitor solution provides a more structured way of managing the shared resource. Instead of directly controlling several synchronization operations throughout the program, the shared state and the synchronization logic are grouped inside the RWMonitor class. This makes the conditions for entering and leaving the critical section easier to organize.

The number of active readers, the writer status, and the number of waiting writers are used to decide which thread can proceed. If a writer is currently active, readers must wait. Similarly, a writer cannot enter while readers are still using the shared resource. The condition variable allows waiting threads to sleep instead of continuously checking the resource.

The writer-preference approach is useful because a continuous stream of new readers should not indefinitely prevent a waiting writer from getting access. When the resource becomes available, waiting threads are notified and check their conditions again before proceeding. The execution trace can therefore show both the waiting and waking behaviour of the monitor implementation.

4. Dining Philosophers (4) - Semaphore Solution

The implementation uses exactly four philosophers and four forks.

Each fork is represented by a semaphore initialized to 1.

Philosopher i uses fork i and fork $(i+1) \% 4$.

An additional semaphore called `room` is initialized to 3. Therefore, at most three philosophers can compete for forks at the same time.

This prevents the circular-wait condition in which all four philosophers hold one fork and wait for the other fork.

Therefore, the implementation prevents deadlock.

Starvation is not completely excluded because Python semaphores do not provide a strict FIFO ordering.

Fork i lies between philosopher i and philosopher $(i+1) \% 4$, so every fork is shared by exactly two neighbours. Each philosopher repeats the cycle thinking, hungry, acquiring forks, eating and releasing forks. Fork ownership is logged immediately after a fork is taken and immediately before it is released, so the HTML always shows which philosopher holds which fork.

The four philosophers repeatedly move through the states of thinking, becoming hungry, acquiring the required forks, eating, and finally releasing the forks. Each fork can be used by only one philosopher at a time because it is represented by a semaphore initialized to 1.

The additional `room` semaphore is important in this implementation. By allowing only three philosophers to compete for forks at a time, the program avoids the situation where all four philosophers each hold one fork and wait indefinitely for the second fork. This provides a simple way to prevent deadlock while still allowing multiple philosophers to compete for resources.

The HTML recording makes the resource allocation easier to observe. When a philosopher obtains a fork, the event is recorded, and when the fork is released, another event is generated. Therefore, the simulation can show the changing ownership of the four forks during the actual execution of the program.

5. Dining Philosophers (4) - Monitor Solution

A Table class is used as the monitor.

The monitor maintains the owner of each of the four forks.

A philosopher can acquire both required forks only when both are available.

The check and acquisition of both forks happen while holding the monitor lock.

A philosopher does not hold one fork while waiting for the other, so the hold-and-wait condition is avoided.

The monitor solution therefore prevents deadlock.

Starvation is theoretically possible if neighbouring philosophers repeatedly get opportunities to eat before another philosopher.

The monitor keeps one owner entry for each fork. A hungry philosopher waits on the condition variable until both neighbouring forks are free. After eating, the philosopher frees both forks and calls `notify_all()` so that waiting philosophers re-check their condition. Because the check and the pickup happen in one step under the lock, two neighbours can never both believe that the same fork is free.

The monitor approach handles the fork allocation differently from the semaphore solution. Instead of allowing a philosopher to acquire the forks individually, the monitor checks whether both required forks are available before giving them to the philosopher. This makes the resource allocation operation controlled as a single synchronized action.

When a philosopher becomes hungry, it waits if either of its required forks is unavailable. Once both forks are free, the monitor assigns them to that philosopher. After the philosopher finishes eating, both forks are released and waiting philosophers are notified.

This design avoids the situation where one philosopher holds one fork while waiting for another. As a result, the hold-and-wait condition required for deadlock is avoided. The ownership information maintained by the monitor also makes it possible for the HTML simulation to display which philosopher currently owns each fork.

6. Event Recording and HTML Generation

The synchronization programs record state changes during actual execution.

Each thread calls functions such as `rec.state(...)` and `rec.fork(...)` whenever its state or resource ownership changes. The recorder stores these events with timing information.

After all threads finish execution, the `write_html()` function creates an HTML file from the recorded events. The events are embedded as data in the generated HTML file, and JavaScript replays the recorded execution in the browser.

For Readers-Writers, the simulation shows reader and writer states, shared-resource activity, and the event sequence.

For Dining Philosophers, the simulation shows all four philosophers, the four forks, philosopher states, and fork ownership.

Therefore, the HTML simulations are generated from actual program execution rather than being separate hand-created animations.

The recorder protects its event list with its own small lock, so events from different threads never overwrite each other, and every event carries a time stamp measured from the start of the run. These time stamps are what appear in the event log of each simulation.

The event-recording mechanism is useful because concurrent execution does not always happen in the same order. Different threads may be scheduled differently during different executions. Recording the actual events allows the generated HTML to represent what happened during that particular execution instead of showing a fixed, manually created sequence.

For the Readers-Writers programs, the recorded information helps identify when readers and writers are waiting, reading, or writing. For the Dining Philosophers programs, the recorded fork events help identify when a philosopher becomes hungry, acquires forks, starts eating, and releases the forks.

The final HTML output therefore acts as a replay of the program execution. It provides a simple visual way to study synchronization behaviour and understand how threads interact with shared resources.

7. Execution Traces

7.1 Readers-Writers Using Semaphore

The execution shows readers entering the reading section and writers obtaining exclusive access to the shared resource.

Example sequence:

0.018s R2 -> waiting

0.018s R2 -> reading (value=0)

0.022s R1 -> waiting

0.022s R1 -> reading (value=0)

0.076s W2 -> waiting

0.118s R2 -> idle

0.122s R1 -> idle

0.122s W2 -> writing (value 0->1)

The exact order varies between executions because the programs use concurrent threads and random delays.

7.2 Readers-Writers Using Monitor

The execution shows readers and writers entering the monitor and waiting when the required condition is not satisfied.

Example sequence:

0.031s W1 -> waiting

0.031s W1 -> writing (value 0->1)

0.097s R1 -> waiting

0.114s R2 -> waiting

0.132s W1 -> idle

0.132s R1 -> reading (value=1)

0.132s R2 -> reading (value=1)

The monitor checks its conditions again after a waiting thread is awakened.

7.3 Dining Philosophers Using Semaphore

The execution shows philosophers moving through different states and acquiring/releasing forks.

Example sequence:

0.059s P2 -> hungry

0.059s P2 -> acquiring

0.059s Fork F2 taken by P2

0.065s Fork F3 taken by P2

0.065s P2 -> eating

0.078s P1 -> hungry

0.078s P1 -> acquiring

0.078s Fork F1 taken by P1

The room = 3 semaphore prevents all four philosophers from simultaneously entering the fork-acquisition stage.

7.4 Dining Philosophers Using Monitor

The execution shows philosophers waiting until both required forks are available.

Example sequence:

0.025s P3 -> hungry

0.026s P3 -> acquiring

0.026s Fork F3 taken by P3

0.026s Fork F0 taken by P3

0.026s P3 -> eating

0.126s P3 -> releasing

0.126s Fork F3 released

The monitor acquires both forks atomically, so a philosopher does not keep one fork while waiting for the other.

8. Race Conditions, Mutual Exclusion, Deadlock and Starvation

Race Condition

A race condition can occur when multiple threads access and modify shared data without proper synchronization. For example, if two writers read the same value before either writes the updated value, one update can be lost.

The Readers-Writers implementations use synchronization mechanisms to prevent simultaneous conflicting access to the shared resource.

Mutual Exclusion

Mutual exclusion ensures that only one writer accesses the shared resource at a time. In the semaphore implementation, `rw_lock` provides this protection. In the monitor implementation, the monitor lock and conditions control access.

In Dining Philosophers, each fork can be owned by only one philosopher at a time.

Deadlock

In Dining Philosophers, deadlock could occur if every philosopher acquired one fork and then waited for the second fork[3].

The semaphore implementation prevents this by allowing only three philosophers to compete for forks at the same time.

The monitor implementation prevents the hold-and-wait situation by acquiring both forks together only when both are available.

Deadlock needs four conditions to hold at the same time: mutual exclusion, hold and wait, no preemption, and circular wait [9]. The semaphore version breaks circular wait, because with `room = 3` at most three philosophers hold forks and at least one of them can always obtain both. The monitor version breaks hold and wait, because a philosopher never holds one fork while waiting for the other.

Starvation

Starvation occurs when a thread or philosopher waits for a long time because other threads repeatedly receive access.

The semaphore Readers-Writers implementation uses readers-preference, so writers can theoretically starve if readers continue to arrive[4].

The monitor Readers-Writers implementation uses writer-preference, which gives waiting writers priority over new readers.

The Dining Philosophers implementations do not provide a strict FIFO fairness guarantee, so starvation is theoretically possible even though deadlock is prevented.

9. Semaphore vs Monitor Comparison

Aspect	Semaphore	Monitor
Structure	Uses synchronization primitives such as semaphores	Combines shared data, a lock, and condition variables
Waiting	Threads block using acquire()	Threads wait using condition variables
Resource control	Programmer explicitly performs acquire() and release() operations	Resource access is controlled inside monitor methods
Error risk	Incorrect acquire()/release() ordering can cause synchronization problems	Encapsulation makes synchronization easier to organize
Level of abstraction	Lower-level synchronization mechanism	Higher-level synchronization abstraction
Conditions	Conditions are usually managed explicitly by the programmer	Conditions are represented using condition variables

In summary, semaphores are flexible but leave the correctness of every acquire and release in the programmer's hands, while a monitor groups the shared data, the lock and the conditions in one place, which makes the rules easier to check [8][10].

10. Screenshots of Generated HTML Simulations

The following images shows outputs pf HTML stimulation:

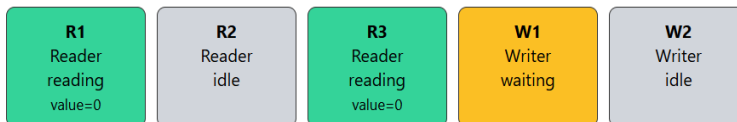
10.1 Readers-Writers Using Semaphore

Readers-Writers (Semaphore)

Semaphores: mutex=1, rw_lock=1 | readers-preference

▶ Start || Pause ⏮ Reset Speed 1 5 / 45

Shared resource: **READ by 2 reader(s)**



Event log (from program trace)

```
0.048s R1 -> waiting
0.048s R1 -> reading (value=0)
0.113s R3 -> waiting
0.113s R3 -> reading (value=0)
0.128s W1 -> waiting
```

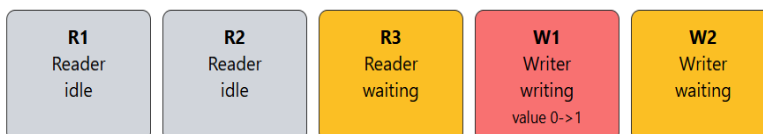
10.1 Readers-Writers Using Monitors

Readers-Writers (Monitor)

Monitor: Condition(lock) | writer-preference

▶ Start || Pause ⏮ Reset Speed 1 4 / 45

Shared resource: **WRITING (exclusive)**



Event log (from program trace)

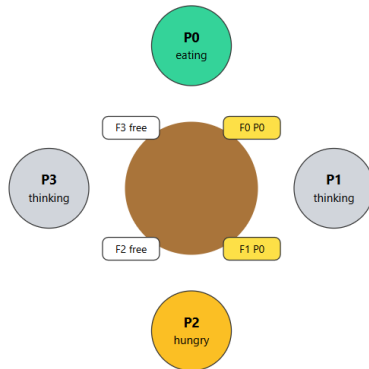
```
0.032s W1 -> waiting
0.032s W1 -> writing (value 0->1)
0.065s W2 -> waiting
0.097s R3 -> waiting
```

10.3 Dining Philosophers Using Semaphore

Dining Philosophers x4 (Semaphore)

Semaphores: fork[0..3]=1 each, room=3 (deadlock avoidance)

▶ Start || Pause ⏮ Reset Speed 1 10 / 148



Event log (from program trace)

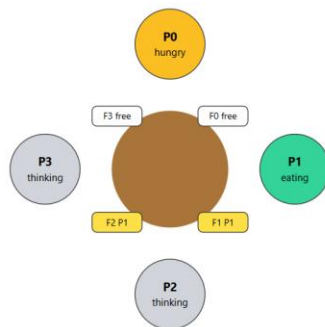
```
0.000s P0 -> thinking
0.000s P1 -> thinking
0.000s P2 -> thinking
0.000s P3 -> thinking
0.033s P0 -> hungry
0.033s P0 -> acquiring
0.033s Fork F0 taken by P0
0.027s Fork F1 taken by P0
```

10.4 Dining Philosophers Using Monitor

Dining Philosophers x4 (Monitor)

Monitor: Condition(lock), both forks taken atomically

▶ Start || Pause ⏮ Reset Speed 1 6 / 144



Event log (from program trace)

```
0.046s P1 -> hungry
0.046s P1 -> acquiring
0.046s Fork F1 taken by P1
0.046s Fork F2 taken by P1
0.046s P1 -> eating
0.094s P0 -> hungry
```

11. AI Usage Summary

Claude (Anthropic) was used during the development of the assignment for planning, code generation, debugging, explanation, and HTML visualization[5].

The important prompts and follow-up prompts used during development are documented in `ai_prompts/prompts.md`.

The generated programs were tested and the synchronization behaviour was checked through actual execution. The final explanations were reviewed to understand the role of semaphores, monitors, locks, condition variables, critical sections, deadlock prevention, and starvation.

One important limitation identified during development is that Python semaphores and condition variables do not provide a strict FIFO fairness guarantee. Therefore, starvation cannot be completely ruled out in all implementations even when deadlock is prevented.

12. References

- [1] Python Software Foundation, "threading — Thread-based parallelism," Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/library/threading.html>
- [2] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts.
- [3] A. S. Tanenbaum and H. Bos, Modern Operating Systems.
- [4] A. B. Downey, The Little Book of Semaphores. [Online]. Available: <https://greenteapress.com/wp/semaphores/>
- [5] Anthropic, "Claude," AI assistant.
- [6] E. W. Dijkstra, "Cooperating sequential processes," Technological University Eindhoven, EWD123, 1965.
- [7] P. J. Courtois, F. Heymans, and D. L. Parnas, "Concurrent control with 'readers' and 'writers'," Communications of the ACM, vol. 14, no. 10, pp. 667–668, 1971.
- [8] C. A. R. Hoare, "Monitors: An operating system structuring concept," Communications of the ACM, vol. 17, no. 10, pp. 549–557, 1974.
- [9] E. G. Coffman, M. J. Elphick, and A. Shoshani, "System deadlocks," ACM Computing Surveys, vol. 3, no. 2, pp. 67–78, 1971.
- [10] W. Stallings, Operating Systems: Internals and Design Principles, Pearson.

13. Academic Integrity Declaration

I declare that I have personally tested and understood the submitted programs. I have disclosed the AI tools and external resources used in developing this assignment. I can explain the synchronization logic, generated HTML simulation, and modifications made to the code. I have not knowingly submitted another student's work or unacknowledged third-party material as my own.

Name: Disha Kumari

USN: NNM24IS074

Signature: _____

Date: _____